

ASSIGNMENT 4

Question:-1

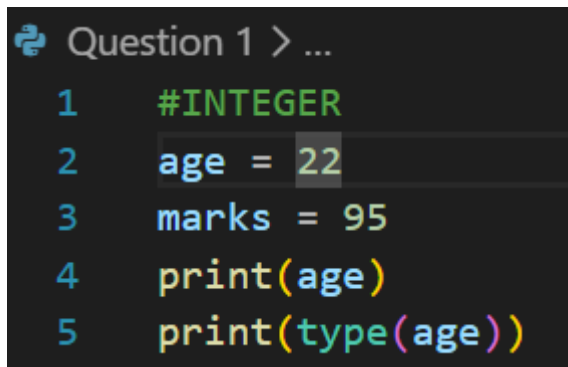
Explain Python Data Types in detail. Discuss the following data types with syntax and examples: • Integer • Float • String • Boolean • List • Tuple • Set • Dictionary

Ans:- A (INTEGER)int:-

An integer is a whole number without a decimal point.
It can be positive, negative, or zero.

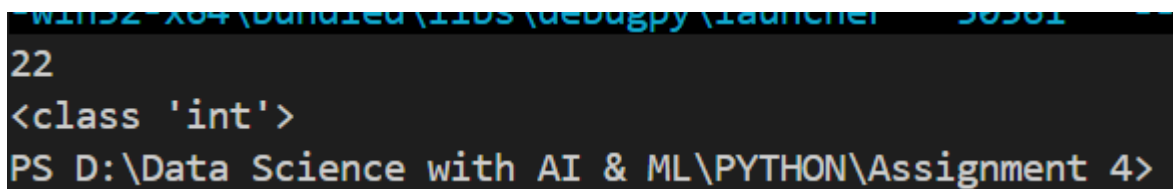
Syntax :- variable_name = integer_value

Example:-



```
Question 1 > ...  
1  #INTEGER  
2  age = 22  
3  marks = 95  
4  print(age)  
5  print(type(age))
```

Output:-



```
22  
<class 'int'>  
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features:-

- Stores whole numbers
- No decimal values
- Unlimited size in Python

B (FLOAT)float:-

A float stores decimal numbers.

Syntax:- variable_name = decimal_number

Example:-

```
# FLOAT
price = 99.99
print(price)
print(type(price))
```

Output:-

```
99.99
<class 'float'>
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features: -

- Used for decimal values
- Supports mathematical operations

C (STRING)string: -

A string is a sequence of characters enclosed in single quotes (' ') or double quotes (" ").

Syntax: -

variable_name = "text"

Example: -

```
# STRING
name = "Ashwani"
message = 'Hello World'

print(name)
print(type(name))
```

```
# String operation
text = "Python"
print(text.upper())
print(text.lower())
print(len(text))
```

Output: -

```
Ashwani
<class 'str'>
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

```
Ashwani
<class 'str'>
PYTHON
python
6
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features: -

- Stores text data.
- Immutable (cannot be changed directly).

D (BOOLEAN)bool: -

Boolean data type stores only two values:

- True
- False

Syntax: -

variable_name = True

Example: -

```
# Boolean
is_student = True
is_logged_in = False
print(is_student)
print(type(is_student))
```

```
age = 22
print(age >= 18)
```

Output: -

```
True
<class 'bool'>
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>

PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
s\LENOVO\AppData\Local\Programs\Python\Python311\python
-win32-x64\bundled\libs\debugpy\launcher' '50914' '--
True
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features

- Used in conditions and decision making.
- Commonly used with if statements.

E (LIST) list: -

A list is an ordered collection of items.

Lists are mutable, meaning elements can be changed.

Syntax: -

```
list_name = [item1, item2, item3]
```

Example: -

```
# LIST
numbers = [10, 20, 30, 40]
print(numbers)
print(type(numbers))
```

```
# LIST OPERATION
students = ["Ashwani", "Anshu", "Anshika"]
students.append("Aman") # Add item
print(students)
print(students[0]) # Indexing
```

Output: -

```
[10, 20, 30, 40]
<class 'list'>
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>

['Ashwani', 'Anshu', 'Anshika', 'Aman']
Ashwani
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features

- Ordered
- Mutable
- Allows duplicate values

F (TUPLE) tuple: -

A tuple is similar to a list, but it is immutable.

Syntax: -

tuple_name = (item1, item2, item3)

Example: -

```
# TUPLE
colors = ("Red", "Green", "Blue")
print(colors)
print(type(colors))

colors = ("Red", "Green", "Blue")
print(colors[1])
```

Output: -

```
('Red', 'Green', 'Blue')
<class 'tuple'>
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>

Green
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features: -

- Ordered
- Immutable
- Faster than lists
- Allows duplicates

G (SET)set: -

A set is an unordered collection of unique elements.

Syntax: -

set_name = {item1, item2, item3}

Example: -

```
# SET
numbers = {1, 2, 3, 4}
print(numbers)
print(type(numbers))
```

```
# SET OPERATION
A = {1, 2, 3}
B = {3, 4, 5}
print(A.union(B))
print(A.intersection(B))
```

Output: -

```
{1, 2, 3, 4}
<class 'set'>
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

```
{1, 2, 3, 4, 5}
{3}
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features: -

- Unordered
- No duplicate values
- Mutable

H (DICTIONARY) dict: -

A dictionary stores data in key-value pairs.

Syntax: -

dict_name = {key1: value1, key2: value2}

Example: -

```
# DICTIONARY
student = {
    "name": "ASHWANI KUMAR",
    "age": 22,
    "course": "Data Science"
}
print(student)
print(type(student))
```

```
# DICTIONARY OPERATION
student = {
    "name": "Anshu",
    "age": 21,
    "course": "Python"
}
print(student["name"])
student["age"] = 20
student["city"] = "Patna"
print(student)
```

Output: -

```
{'name': 'ASHWANI KUMAR', 'age': 22, 'course': 'Data Science'}  
<class 'dict'>  
PS D:\Data Science with AI & ML\PYTHON\Assignment 4> ^C
```

```
Anshu  
{'name': 'Anshu', 'age': 20, 'course': 'Python', 'city': 'Patna'}  
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Features: -

- Stores data in key-value form
- Mutable
- Keys must be unique

Question: - 2

Write a Python program to demonstrate dynamic typing and type checking using the type() function. The program should: ● Declare variables of multiple data types ● Print their values ● Print their corresponding data types

Ans: - Python is a dynamically typed language, which means we do not need to declare the data type of a variable explicitly.

Python automatically identifies the data type based on the value assigned to the variable.

The type () function is used to check the data type of a variable.

Example: -

Type function

Integer

a = 10

Float

b = 5.5

String

c = "Hello Python"

```
# Boolean
d = True

# List
e = [1, 2, 3]

# Tuple
f = (10, 20, 30)

# Set
g = {1, 2, 3}

# Dictionary
h = {"name": "Ansh", "age": 21}

# Printing values and their data types
print("Value of a:", a)
print("Data Type of a:", type(a))
print("Value of b:", b)
print("Data Type of b:", type(b))
print("Value of c:", c)
print("Data Type of c:", type(c))
print("Value of d:", d)
print("Data Type of d:", type(d))
print("Value of e:", e)
print("Data Type of e:", type(e))
print("Value of f:", f)
print("Data Type of f:", type(f))
print("Value of g:", g)
print("Data Type of g:", type(g))
print("Value of h:", h)
print("Data Type of h:", type(h))
```

Output: -

```
Value of a: 10
Data Type of a: <class 'int'>
Value of b: 5.5
Data Type of b: <class 'float'>
Value of c: Hello Python
Data Type of c: <class 'str'>
Value of d: True
Data Type of d: <class 'bool'>
Value of e: [1, 2, 3]
Data Type of e: <class 'list'>
Value of f: (10, 20, 30)
Data Type of f: <class 'tuple'>
Value of g: {1, 2, 3}
Data Type of g: <class 'set'>
Value of h: {'name': 'Ansh', 'age': 21}
Data Type of h: <class 'dict'>
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Question: - 3

Differentiate between Mutable and Immutable Data Types in Python with suitable examples. Also explain: ● Why strings are immutable ● Why lists are mutable ● Real-time use cases of both

Ans: - **Mutable vs Immutable Data Types in Python**

In Python, data types are divided into two categories:

1. **Mutable Data Types** → Can be changed after creation
2. **Immutable Data Types** → Cannot be changed after creation

Difference Between Mutable and Immutable Data Types

Feature	Mutable	Immutable
Meaning	Value can be modified	Value cannot be modified

Feature	Mutable	Immutable
Memory	Same memory location used after changes	New memory location created after changes
Changes Allowed	Yes	No
Examples	List, Set, Dictionary	Integer, Float, String, Tuple, Boolean

1. Mutable Data Types

Mutable objects can be changed after they are created.

Example: - List

```
numbers = [10, 20, 30]
print("Original List:", numbers)
numbers[1] = 50
print("Modified List:", numbers)
```

Output: -

```
Original List: [10, 20, 30]
Modified List: [10, 50, 30]
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

2. Immutable Data Types

Immutable objects cannot be changed after creation.

Example: - String

```
name = "Python"
print(name)
name[0] = "J" name = 'Python'
```

Output: -

```
TypeError: 'str' object does not support item assignment
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Why Strings are Immutable

Strings are immutable because:

- Security and data integrity are maintained
- Faster performance due to memory optimization
- Strings are heavily used in Python programs

Example: -

```
text = "Hello"  
new_text = text.replace("H", "J")  
print(text)  
print(new_text)
```

Output: -

```
Hello  
Jello  
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Why Lists are Mutable

Lists are mutable because:

- Data often needs modification during program execution
- Easy insertion, deletion, and updating of elements
- Useful for dynamic data storage

Example: -

```
fruits = ["Apple", "Banana"]  
fruits.append("Mango")  
print(fruits)
```

Output: -

```
['Apple', 'Banana', 'Mango']  
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Real-Time Use Cases of Immutable Data Types

1. Passwords and Security

Strings are immutable, which helps keep data secure.

Example:

```
password = "admin123"
```

2. Fixed Data

Tuples are used for fixed values like coordinates.

Example:

```
location = (28.61, 77.20)
```

3. Dictionary Keys

Immutable types can be used as dictionary keys.

Example:

```
student = {  
    "name": "Ansh"  
}
```

Real-Time Use Cases of Mutable Data Types

1. Student Records

Lists are used when records need updates.

Example:

```
students = ["Ansh", "Rahul"]  
students.append("Priya")
```

2. Shopping Cart

Items can be added or removed dynamically.

Example:

```
cart = ["Shoes", "Watch"]  
cart.remove("Watch")
```

3. Database-like Storage

Dictionaries store editable user information.

Example:

```
user = {  
    "name": "Ansh",  
    "age": 21  
}
```

```
user["age"] = 22
```

Question: - 4

Write a Python program to perform various operations on Python collections: ● List operations (append() , remove() , slicing) ● Tuple indexing ● Set operations (union , intersection) ● Dictionary operations (keys() , values() , items())

Ans: -

Operation

List Operations

```
numbers = [10, 20, 30, 40]
```

```
print("Original List:", numbers)
```

append()

```
numbers.append(50)
```

```
print("After append():", numbers)
```

remove()

```
numbers.remove(20)
```

```
print("After remove():", numbers)
```

slicing

```
print("List Slicing:", numbers[1:4])
```

Tuple Indexing

```
colors = ("Red", "Green", "Blue", "Yellow")
```

```
print("\nTuple:", colors)

# Indexing
print("First Element:", colors[0])
print("Second Element:", colors[1])


# Set Operations
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print("\nSet A:", A)
print("Set B:", B)

# Union
print("Union:", A.union(B))

# Intersection
print("Intersection:", A.intersection(B))


# Dictionary Operations
student = {
    "name": "Ansh",
    "age": 21,
    "course": "Python"}
print("\nDictionary:", student)

# keys()
print("Keys:", student.keys())

# values()
print("Values:", student.values())

# items()
print("Items:", student.items())
```

Output: -

```
Original List: [10, 20, 30, 40]
After append(): [10, 20, 30, 40, 50]
After remove(): [10, 30, 40, 50]
List Slicing: [30, 40, 50]

Tuple: ('Red', 'Green', 'Blue', 'Yellow')
First Element: Red
Second Element: Green

Set A: {1, 2, 3, 4}
Set B: {3, 4, 5, 6}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}

Dictionary: {'name': 'Ansh', 'age': 21, 'course': 'Python'}
Keys: dict_keys(['name', 'age', 'course'])
Values: dict_values(['Ansh', 21, 'Python'])
Items: dict_items([('name', 'Ansh'), ('age', 21), ('course', 'Python')])
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

Explanation

1. List Operations

- `append()` → Adds an element at the end of the list
- `remove()` → Removes a specific element
- slicing → Extracts a portion of the list

2. Tuple Indexing

- Access tuple elements using index numbers
- Index starts from 0

3. Set Operations

- `union()` → Combines all unique elements
- `intersection()` → Finds common elements

4. Dictionary Operations

- `keys()` → Returns all keys
- `values()` → Returns all values
- `items()` → Returns key-value pairs

Question: - 5

Develop a mini-Student Management System using Python data types. The program should: ● Store student details using Dictionary ● Store subject marks using List ● Calculate total and average marks ● Display the output in proper format

Ans: -

```
# Mini Student Management System
# Student details stored in Dictionary
student = {
    "name": "Ashwani Kumar",
    "roll_no": 2,
    "course": "MCA"
}
# Subject marks stored in List
marks = [85, 90, 78, 88, 92]
# Calculate total marks
total = sum(marks)
# Calculate average marks
average = total / len(marks)
# Display output
print("----- Student Details -----")
print("Name      :", student["name"])
print("Roll No   :", student["roll_no"])
print("Course    :", student["course"])
print("\nSubject Marks :", marks)
print("\nTotal Marks   :", total)
print("Average Marks :", average)
```

Output: -

```
----- Student Details -----
Name      : Ashwani Kumar
Roll No   : 2
Course    : MCA

Subject Marks : [85, 90, 78, 88, 92]

Total Marks   : 433
Average Marks : 86.6
PS D:\Data Science with AI & ML\PYTHON\Assignment 4>
```

